

Big Data Programming

Python & Development Environment

Basic Syntax

출력하기: print()

hello.py

C: > Users > 박영재 > Downloads > BigDataProgramming > hello.py

```
1  print("Hello")
2  print("Python")
```

Problems Output Debug Console Terminal Ports powershell

```
PS C:\Users\박영재> cd C:\Users\박영재\Downloads\BigDataProgramming
PS C:\Users\박영재\Downloads\BigDataProgramming> python hello.py
Hello
Python
PS C:\Users\박영재\Downloads\BigDataProgramming>
```

괄호 안의 식을 먼저 계산하고 그 결과를 출력한다.
기본 설정에서는 출력할 때마다 마지막에 줄바꿈이 붙는다.

Running Python

Quick Review

Q1. Python과 VS Code의 역할은 어떻게 다른가?

Python은 프로그래밍 언어이며, Python 인터프리터가 작성한 코드를 실행한다. VS Code는 코드를 작성하고 파일을 관리하는 편집기이다. 편집기를 설치한 것과 Python 실행 환경을 설치한 것은 구분해야 한다.

Q2. Interactive Mode는 언제 사용하는가?

짧은 계산이나 문법을 바로 확인할 때 사용한다. 여러 줄의 코드를 저장하고 반복해서 실행하려면 Script Mode가 편리하다. 같은 Python을 사용하지만 코드를 입력하고 관리하는 방식이 다르다.

Q3. Python 소스 파일은 어떻게 저장하고 실행하는가?

Python 소스 파일의 확장자는 .py이다. VS Code에서 hello.py를 작성하고 저장한 뒤 실행한다. 파일을 수정했다면 저장한 파일이 맞는지, 실행한 파일의 이름과 위치가 맞는지 함께 확인한다.

Basic Syntax

출력하기: print()

print()의 역할

print()는 괄호 안에 전달한 값을 출력하는 내장 함수이다. 문자열뿐 아니라 숫자, 변수의 값, 계산 결과도 출력할 수 있다. 괄호 안의 식을 먼저 계산하고 그 결과를 출력한다.

코드의 순서와 출력의 순서

현재와 같은 직선적인 코드는 위에서 아래로 실행된다. print("Hello")가 먼저 실행되고 다음 줄의 print("Python")이 실행된다. 기본 설정에서는 출력할 때마다 마지막에 줄바꿈이 붙는다.

계산과 출력은 구분한다

스크립트에 10 + 20만 적으면 계산은 하지만 결과를 화면에 표시하지 않는다. 결과를 보려면 print(10 + 20)처럼 출력 명령을 작성한다. Interactive Mode의 자동 표시와 구분해야 한다.

hello.py

```
print("Hello")
print("Python")
```

실행 결과

```
Hello
Python
```

계산한 값을 출력하기

```
print(10 + 20)
print("Python", 2026)
```

첫 줄은 30, 다음 줄은 Python 2026을 출력한다.

Basic Syntax

문자열과 출력 형식 이해하기

따옴표 안과 밖은 다르게 해석한다

"10 + 20"은 계산식이 아니라 문자로 이루어진 문자열이다. 반면 따옴표 없는 10 + 20은 덧셈 표현식이다. 출력할 내용 자체를 보여줄지, 계산한 결과를 보여줄지 구분한다.

여러 값과 줄바꿈

쉼표로 여러 값을 전달하면 기본적으로 값 사이에 공백이 들어간다. print()처럼 아무 값도 전달하지 않으면 빈 줄을 출력한다. 문자열 안의 \n은 줄바꿈을 나타낸다.

출력 모양을 바꾸는 선택 사항

sep은 값 사이의 구분 문자열, end는 출력 마지막에 붙일 문자열이다. 기본 공백과 줄바꿈을 바꾸는 기능으로 이해한다. 출력 모양을 바꿔도 원래 값의 자료형이 바뀌지는 않는다.

계산식과 문자열 비교

```
print(10 + 20)
print("10 + 20")
```

출력: 30, 다음 줄에 10 + 20

여러 값과 줄바꿈

```
print("Python", 2026)
print()
print("Hello\nPython")
```

구분 문자열 지정

```
print("A", "B", sep="-")
print("Hello", end=" ")
print("Python")
```

출력: A-B, 다음 줄에 Hello Python

Basic Syntax

주석 (Comments)

주석은 설명을 남기는 문법이다

문자열 밖에서 #으로 시작한 내용은 해당 줄의 끝까지 주석으로 처리된다. 주석은 계산하거나 출력할 명령이 아니라, 코드를 읽는 사람에게 목적과 판단 근거를 전달하는 설명이다.

어디에 주석을 작성할 수 있는가?

한 줄 전체를 설명으로 작성할 수도 있고, 실행할 코드 뒤에 짧게 덧붙일 수도 있다. 줄의 일부를 주석으로 바꾸면 그 부분은 실행되지 않는다. 주석을 추가한 뒤에는 실제로 실행되는 코드가 무엇인지 확인한다.

좋은 주석은 코드의 의미를 보완한다

print를 사용했다는 사실만 반복하기보다 무엇을 확인하려고 출력하는지 적는 편이 유용하다. 코드가 바뀌면 주석도 함께 수정한다. 오래된 설명은 오히려 코드를 잘못 이해하게 만들 수 있다.

코드 예제

```
# This is a comment.
print("Hello")  # 인사말 출력
```

실행 결과

```
Hello
```

문자열 안의

```
print("#은 여기서 문자이다.")
```

문자열 내부의 #은 주석의 시작이 아니라 출력할 문자이다.

Basic Syntax

주석 (Comments)

hello.py

C: > Users > 박영재 > Downloads > BigDataProgramming > hello.py

```
1   # This is a comment.
2   print("Hello")    # This is also a comment.
```

Problems Output Debug Console Terminal Ports powershell

```
PS C:\Users\박영재> cd C:\Users\박영재\Downloads\BigDataProgramming
PS C:\Users\박영재\Downloads\BigDataProgramming> python hello.py
Hello
PS C:\Users\박영재\Downloads\BigDataProgramming>
```

문자열 밖에서 #으로 시작한 내용은 해당 줄의 끝까지 주석으로 처리된다.
주석은 코드를 읽는 사람에게 목적과 판단 근거를 전달하는 설명이다.

Errors

Errors Are Part of Programming

문법을 확인하는 단계에서 생기는 문제

괄호나 따옴표가 닫히지 않았거나 필요한 기호가 빠지면 Python이 코드를 문법적으로 분석하지 못할 수 있다. 이런 문제는 SyntaxError로 보고된다. 먼저 문장의 구조가 완성되었는지 살펴본다.

코드를 실행하는 단계에서 생기는 문제

문법이 맞더라도 실제 값으로 연산하는 과정에서 문제가 발생할 수 있다. 문자열과 정수를 더하거나, 0으로 나누거나, 정의되지 않은 이름을 사용한 경우가 여기에 해당한다.

오류 메시지가 없다고 정답은 아니다

프로그램이 끝까지 실행되어도 계산식이나 조건을 잘못 작성하면 의도와 다른 결과가 나온다. 오류 메시지를 읽는 일과 계산 결과가 맞는지 검증하는 일은 모두 필요하다.

이번 시간에 구분할 오류

SyntaxError
문법적으로 올바른 코드인지 확인한다.

TypeError
연산에 사용한 자료형을 확인한다.

ZeroDivisionError
나누는 값이 0인지 확인한다.

NameError
사용한 이름이 먼저 정의되었는지 확인한다.

학습을 위해 발생 시점으로 나눈 설명이다. SyntaxError도 Python 예외 클래스의 하나이다.

Errors

Syntax Error

문장의 구조가 완성되어야 한다

print("Hello"에는 여는 괄호만 있고 닫는 괄호가 없다. Python은 함수 호출이 어디서 끝나는지 확정할 수 없으므로 문법 오류를 알린다. 따옴표와 괄호의 짝을 함께 확인한다.

기호 하나가 문장의 의미를 결정한다

if 조건 뒤에는 콜론이 필요하다. 오른쪽 예제에서는 조건 뒤의 :가 빠져 있다. 지금은 if의 자세한 기능보다 문법에 필요한 기호가 누락되면 실행을 시작할 수 없다는 점에 집중한다.

표시된 줄이 원인의 전부는 아니다

오류 위치는 Python이 문제를 발견한 위치이다. 실제 원인은 앞줄의 닫히지 않은 괄호일 수도 있다. 현재처럼 한 .py 파일을 직접 실행할 때 문법 분석에 실패하면 그 파일의 본문은 실행되지 않는다.

닫는 괄호가 없는 예

```
print("Hello"
SyntaxError: '(' was never closed
```

수정

```
print("Hello")
```

콜론이 없는 예

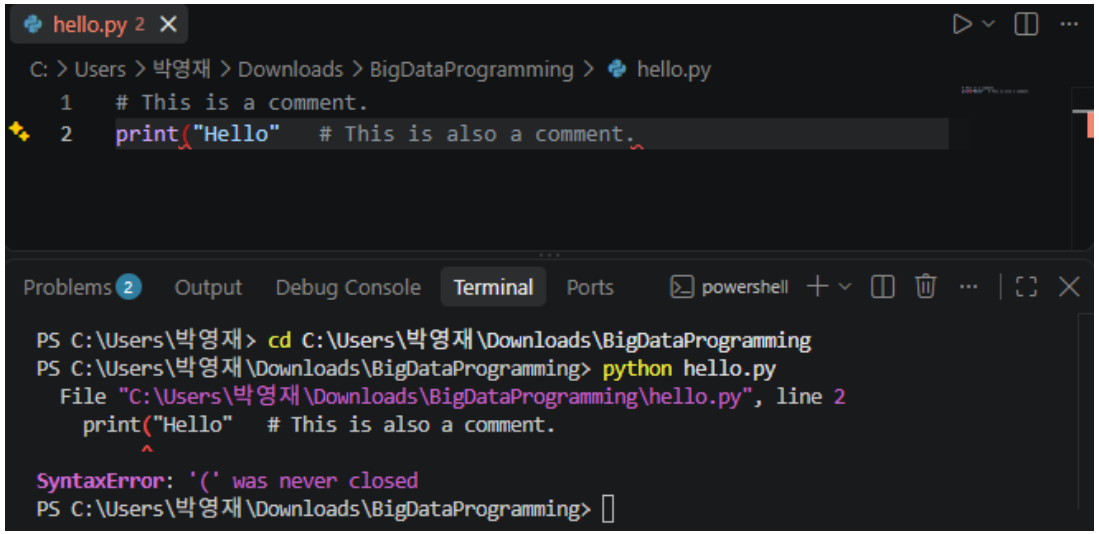
```
if 3 > 2
    print("Yes")
```

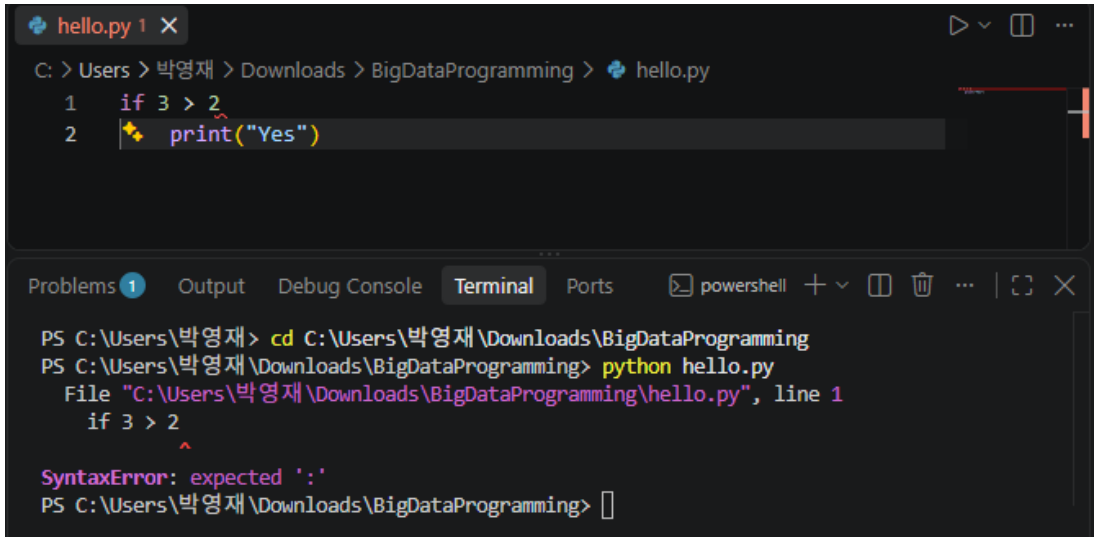
수정

```
if 3 > 2:
    print("Yes")
```

Errors

Syntax Error





오류 위치는 Python이 문제를 발견한 위치이다.
실제 원인은 앞줄의 닫히지 않은 괄호일 수도 있다.

Errors

TypeError

자료형에 맞는 연산인지 확인한다

"Hi"는 문자열이고 3은 정수이다. 이 예제에서 +는 문자열끼리 연결하는 연산으로 해석되지만 오른쪽 값은 문자열이 아니다. Python은 정수를 임의로 문자열로 바꾸어 연결하지 않는다.

원하는 결과에 따라 수정 방법이 달라진다

Hi3을 만들려면 str(3)으로 문자열로 변환한 뒤 연결한다. Hi 3처럼 두 값을 나누어 표시하려면 print("Hi", 3)을 사용한다. 오류가 사라진다는 이유만으로 목적과 다른 코드를 선택해서는 안 된다.

같은 기호도 자료형에 따라 동작이 다르다

숫자의 +는 덧셈이고 문자열의 +는 연결이다. 문자열에 정수를 곱하는 "Hi" * 3은 세 번 반복한 문자열을 만든다. 이것은 Hi3을 만드는 수정 방법과는 결과가 다르다.

문제가 있는 코드

```
print("Hi" + 3)
```

TypeError: can only concatenate str (not "int") to str

문자열로 변환하여 연결

```
print("Hi" + str(3))
```

출력: Hi3

서로 다른 값을 함께 출력

```
print("Hi", 3)
```

출력: Hi 3

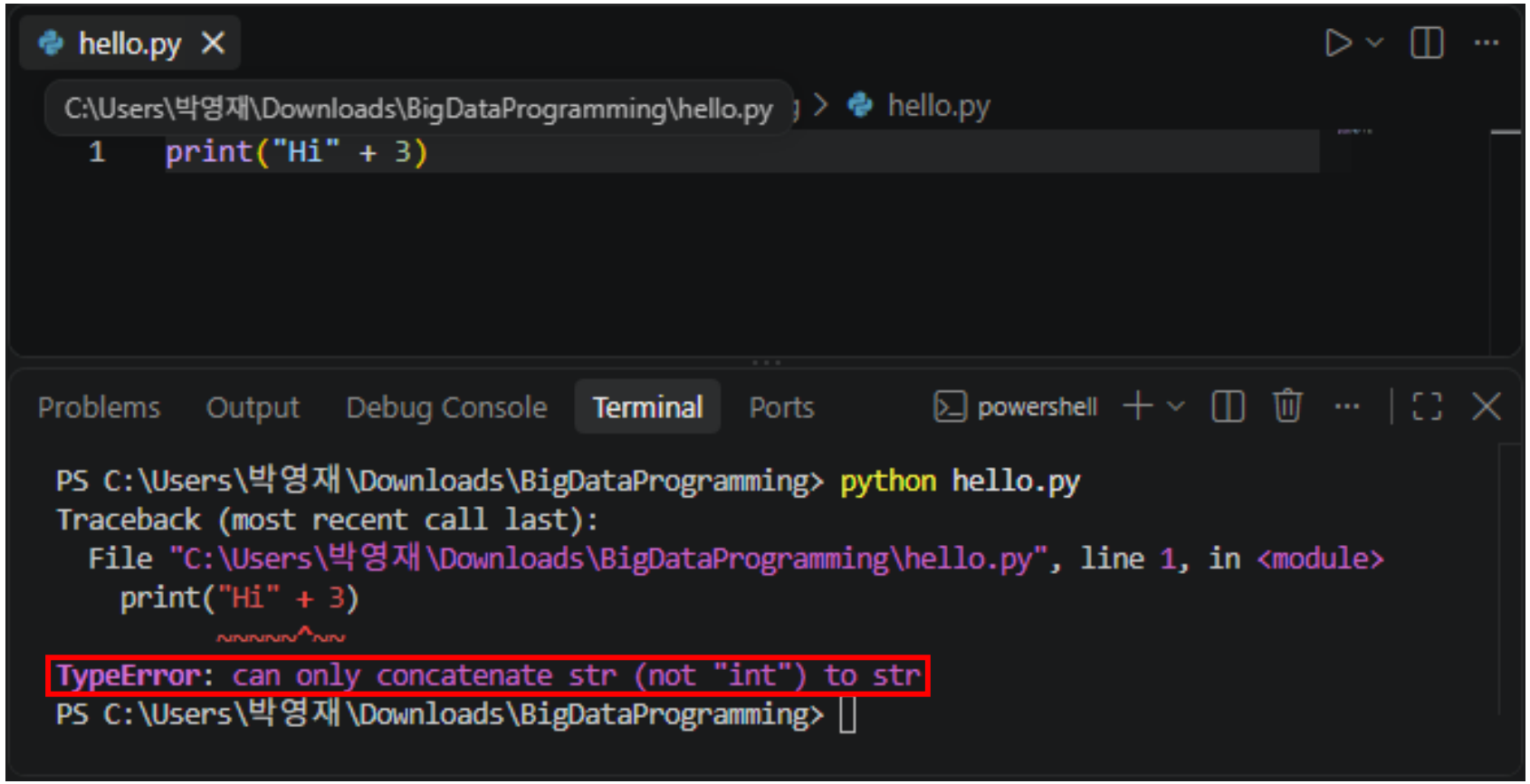
문자열 반복은 다른 연산이다

```
print("Hi" * 3)
```

출력: HiHiHi

Errors

TypeError



"Hi"는 문자열이고 3은 정수이다. Python은 정수를 임의로 문자열로 바꾸어 연결하지 않는다.

Errors

ZeroDivisionError

나누는 값이 0이면 계산할 수 없다

10 / 0을 실행하면 ZeroDivisionError가 발생한다. 나누는 값이 코드에 직접 적혀 있지 않아도 결과는 같다. 변수에 0이 들어 있거나 계산 결과가 0이 되면 같은 문제가 생긴다.

기호보다 실제 값을 확인한다

10 / count가 문법적으로 맞다는 사실만으로 안전하게 실행된다고 판단할 수 없다. 해당 줄을 실행하는 시점의 count 값을 확인해야 한다. print()로 연산 직전의 값을 확인하면 원인을 찾는 데 도움이 된다.

0을 다른 값으로 바꾸는 것이 항상 해결은 아니다

평균을 구하는데 자료가 하나도 없다면 개수가 0일 수 있다. 이를 임의로 1로 바꾸면 프로그램은 실행되어도 계산의 의미가 달라진다. 자료가 없는 상황을 어떻게 처리할지 먼저 정해야 한다.

코드 예제

```
print(10 / 0)
ZeroDivisionError: division by zero
```

변수에 0이 들어 있는 경우

```
count = 0
print(count)
print(10 / count)
```

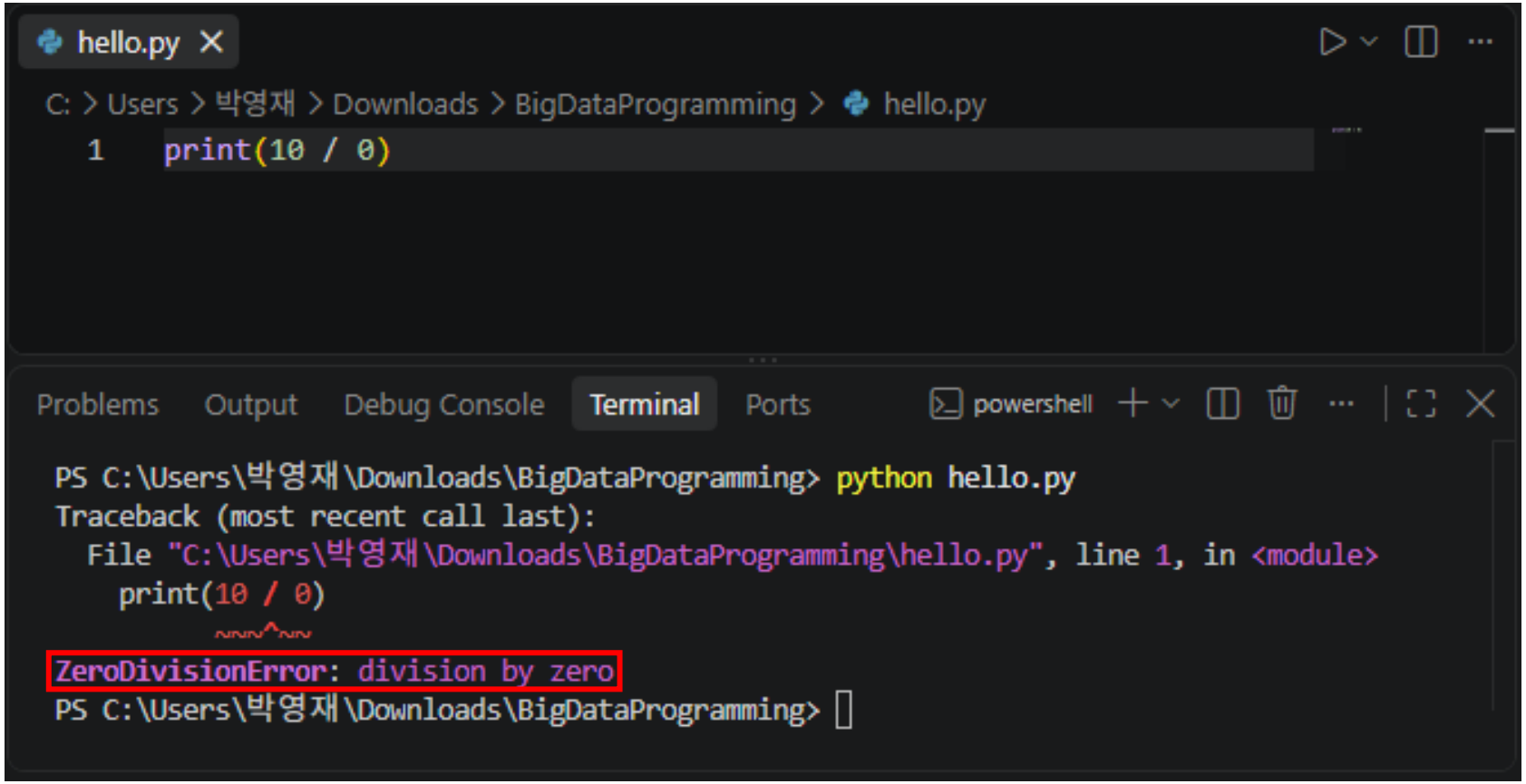
계산한 결과가 0인 경우

```
count = 3 - 3
print(10 / count)
```

세 경우 모두 나누는 값이 0이다. 예외를 처리하지 않으면 실행이 중단된다.

Errors

ZeroDivisionError



나누는 값이 코드에 직접 적혀 있지 않아도 결과는 같다.
변수에 0이 들어 있거나 계산 결과가 0이 되면 같은 문제가 생긴다.

Errors

NameError

이름을 사용하기 전에 정의해야 한다

print(score)를 실행하려면 Python이 score라는 이름에 연결된 값을 찾을 수 있어야 한다. 아직 score에 값을 대입하지 않았다면 이름을 찾지 못해 NameError가 발생한다.

대소문자와 철자는 이름의 일부이다

score와 Score는 다른 이름이다. score = 95를 작성해도 Score는 만들어지지 않는다. print를 pront로 적은 경우도 이름을 호출하는 문법은 맞지만, pront가 정의되지 않았다면 NameError가 발생한다.

실행 환경도 확인한다

Interactive Mode에서 만든 변수가 별도의 스크립트 실행에 자동으로 전달되는 것은 아니다. 현재 파일에도 필요한 대입문이 있어야 하며, 그 대입이 사용 전에 실행되어야 한다.

아직 정의하지 않은 이름

```
print(score)
NameError: name 'score' is not defined
```

먼저 값을 대입한 경우

```
score = 95
print(score)
```

출력: 95

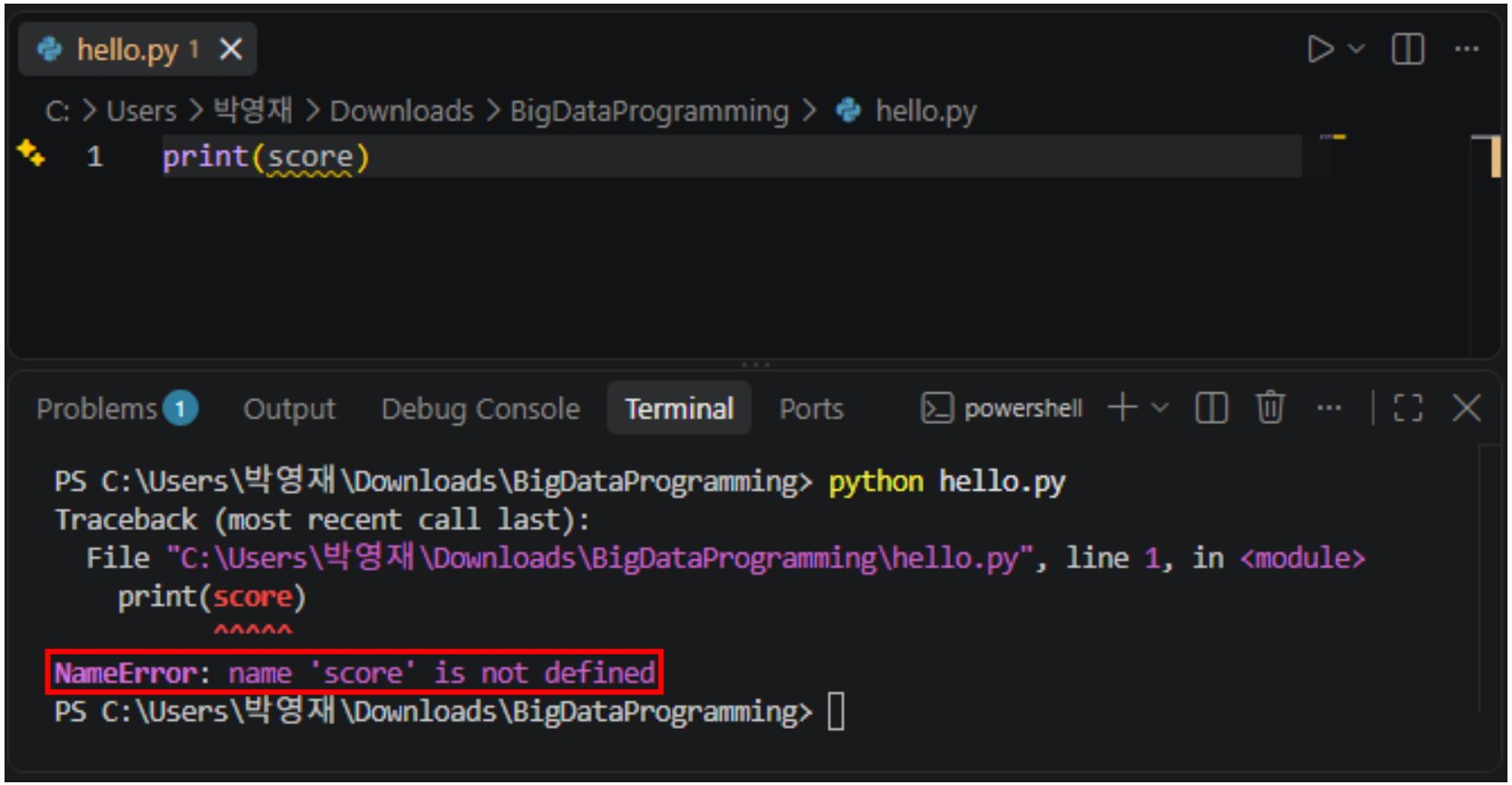
대소문자를 다르게 쓴 경우

```
score = 95
print(Score)
```

Score가 별도로 정의되지 않았다면 NameError가 발생한다.

Errors

NameError



아직 score에 값을 대입하지 않았다면 이름을 찾지 못해 NameError가 발생한다.
score와 Score는 다른 이름이다.

Errors

Traceback과 실행 순서 읽기

마지막 줄에서 오류 종류를 확인한다

기본적인 Traceback의 마지막 오류 줄에는 예외 이름과 설명이 나온다. 앞부분에는 파일과 줄 번호, 문제가 발생한 코드가 제시된다. 단순히 빨간 글씨 전체를 읽기보다 종류와 위치를 나누어 확인한다.

오류 이전과 이후의 실행을 구분한다

Start를 출력한 뒤 다음 줄의 나눗셈에서 예외가 발생한다. 예외를 처리하는 코드가 없어 End를 출력하는 줄까지 진행하지 못한다. 이미 출력한 Start가 취소되는 것은 아니다.

수정한 다음에는 다시 실행한다

오류를 수정하고 저장한 뒤 다시 실행한다.
터미널에서 이전 출력과 새 출력을 구분한다.
오류가 사라졌는지뿐 아니라 예상한 출력이 모두 나왔는지도 확인한다.

실행할 코드

```
print("Start")
print(10 / 0)
print("End")
```

출력과 오류의 핵심 부분

```
Start
ZeroDivisionError: division by zero
```

Traceback에서 찾을 정보

예외 이름은 무엇인가?
어느 파일의 몇 번째 줄인가?
해당 줄에서 어떤 값을 사용했는가?

오류가 발생하기 전까지의 출력과 실행되지 않은 부분을 구분한다.

Errors

오류를 분류하고 의도에 맞게 수정하기

수정 전에 먼저 원인을 말해 본다

오류가 보인다고 기호나 숫자를 무작정 바꾸지 않는다. 문법 문제인지, 자료형 문제인지, 값의 문제인지, 이름의 문제인지 구분하면 확인할 범위를 줄일 수 있다.

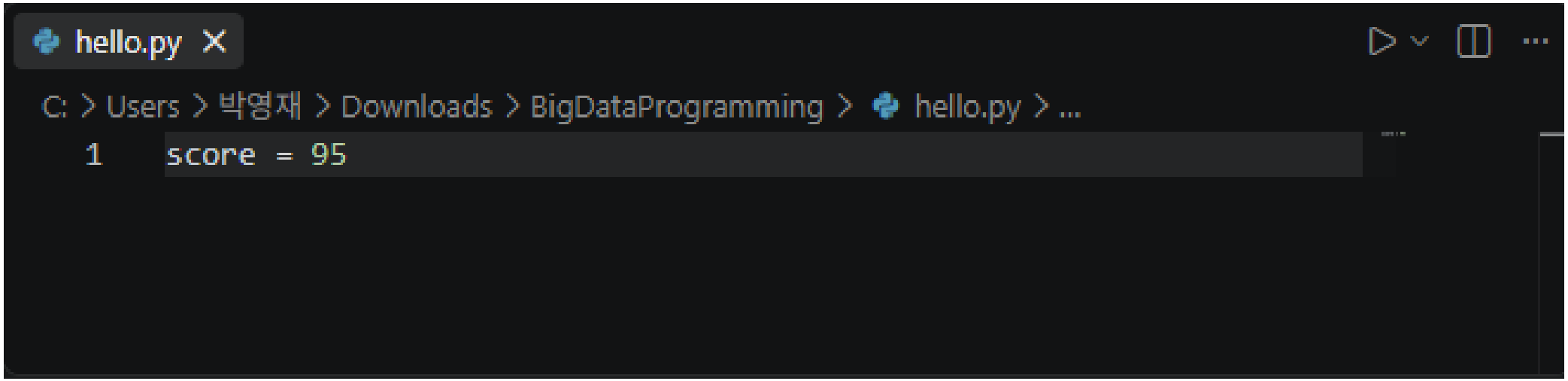
문제가 있는 코드	예상 오류	확인할 내용
print("Hello"	SyntaxError	닫는 괄호가 있는가?
print("Hi" + 3)	TypeError	두 값의 자료형은 무엇인가?
print(10 / 0)	ZeroDivisionError	분모가 왜 0이 되었는가?
print(score)	NameError	score를 먼저 정의했는가?

수정의 성공 기준

오류가 사라졌다는 사실만으로 수정이 끝난 것은 아니다. 원래 출력하려던 값이 나오는지 확인하고, 입력 값이 달라져도 설명이 가능한지 생각한다. 특히 이름이나 숫자를 임의로 바꾼 수정은 계산 목적을 바꿀 수 있다.

Variables

변수란 무엇인가? (Variable)



score : name
95 : value
= : assignment

score는 이름이고, 95는 값이며, int는 해당 객체의 자료형이다.
이름만으로 자료형이나 값이 결정되지는 않는다.

Variables

변수란 무엇인가? (Variable)

값을 다시 사용하기 위해 이름을 붙인다

프로그램에서는 계산한 값을 다른 계산이나 출력에 다시 사용한다. score = 95는 정수 객체 95를 score라는 이름으로 가리키도록 연결한다. 이후 score를 사용하면 그 이름에 연결된 값을 참조한다.

이름과 객체를 구분한다

score는 이름이고, 95는 값이며, int는 해당 객체의 자료형이다. 변수 이름이 숫자 자체인 것은 아니다. 서로 다른 이름이 같은 객체를 가리킬 수도 있으며, 한 이름을 다른 객체에 다시 연결할 수도 있다.

변수명은 의미를 전달한다

x보다 score라는 이름이 값의 역할을 설명하기 쉽다. 다만 이름만으로 자료형이나 값이 결정되지는 않는다. score라는 이름에 무엇이 대입되었는지는 실제 코드를 확인해야 한다.

코드 예제

```
score = 95
print(score)
```

각 요소의 의미

score　값을 참조하는 이름
95　정수 값
=　이름에 값을 연결하는 대입

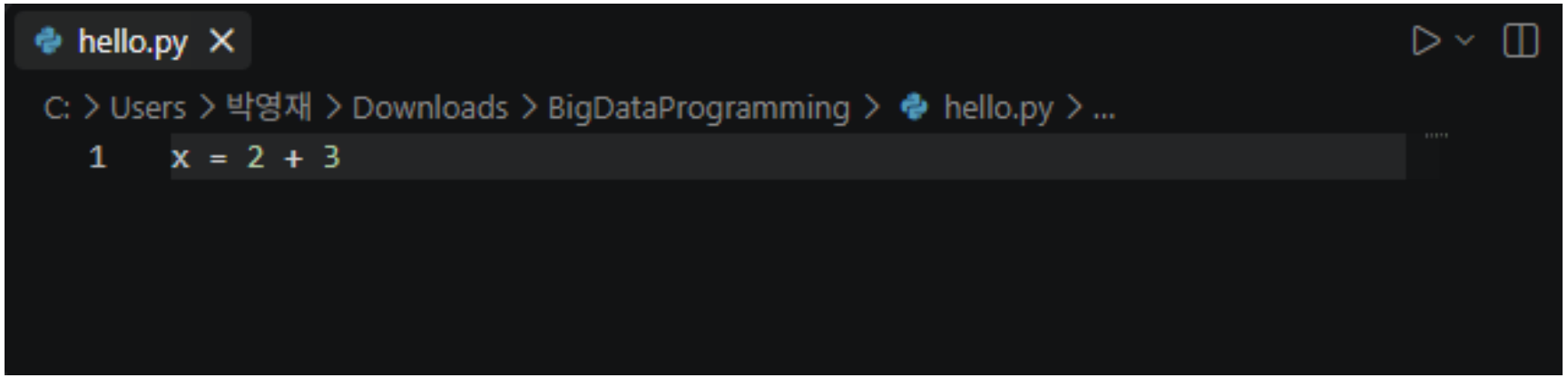
객체를 참조하는 관계

score → 95

95는 int 객체이다. score라는 이름 자체가 int에 고정되는 것은 아니다.

Variables

대입은 어떻게 실행될까? (Assignment)



① Right-hand side 계산

2 + 3 = 5

② 이름에 연결

x ——▶ 5

오른쪽의 2 + 3을 계산하여 5를 얻는다.
그다음 왼쪽 이름 x가 그 결과를 가리키도록 연결한다.

Variables

대입은 어떻게 실행될까? (Assignment)

먼저 오른쪽 표현식을 평가한다

x = 2 + 3에서 Python은 오른쪽의 2 + 3을 계산하여 5를 얻는다. 그다음 왼쪽 이름 x가 그 결과를 가리키도록 연결한다. 오른쪽의 계산이 실패하면 이 대입으로 새 결과가 연결되지 않는다.

왼쪽과 오른쪽의 역할이 다르다

현재 배우는 이름 대입에서 왼쪽은 결과를 연결할 이름이고 오른쪽은 계산할 값이나 표현식이다. 5 = x처럼 값 자체에 이름을 대입하는 문장은 작성할 수 없다. =는 두 식이 같다는 수학 기호가 아니다.

대입과 출력은 별개의 명령이다

x = 2 + 3만 실행하면 x는 5를 가리키지만 화면에는 자동으로 출력되지 않는다. 값을 확인하려면 print(x)를 추가한다. 아무 출력이 없다고 대입이 실행되지 않았다고 판단하지 않는다.

대입문

x = 2 + 3

실행 과정

1. 오른쪽의 2 + 3을 계산한다.
2. 계산 결과 5를 얻는다.
3. 이름 x를 그 결과에 연결한다.

x → 5

값을 확인하는 명령

print(x)

출력: 5

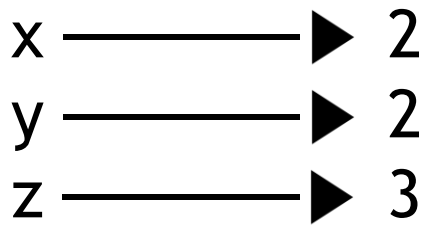
Variables

Assignment Example

hello.py

C: > Users > 박영재 > Downloads > BigDataProgramming > hello.py > ...

```
1   x = 2
2   y = x
3   z = x + 1
```



y = x는 현재 x가 가리키는 객체에 y도 연결한다.
z에 x + 1이라는 계산식이 그대로 저장되는 것은 아니다.

Variables

Assignment Example

각 줄은 그 시점의 값을 사용한다

x = 2가 실행되면 x는 2를 가리킨다. y = x는 현재 x가 가리키는 객체에 y도 연결한다. x라는 글자 자체를 저장하거나, x를 계속 추적하는 규칙을 만드는 것은 아니다.

표현식의 계산 결과를 연결한다

z = x + 1은 현재 x의 값 2에 1을 더한 결과 3을 z에 연결한다. z에 x + 1이라는 계산식이 그대로 저장되는 것은 아니다. 이후의 계산에서는 이미 구해진 z의 값을 사용한다.

변수 사이에 자동 갱신 관계가 생기지 않는다

스프레드시트 수식처럼 x를 바꿀 때마다 y나 z가 자동으로 재계산된다고 생각하지 않는다.
Python의 대입문은 실행된 시점에 한 번 평가한다. 다시 계산하려면 해당 대입문을 다시 실행해야 한다.

코드 예제

```
x = 2
y = x
z = x + 1
```

실행 후의 값

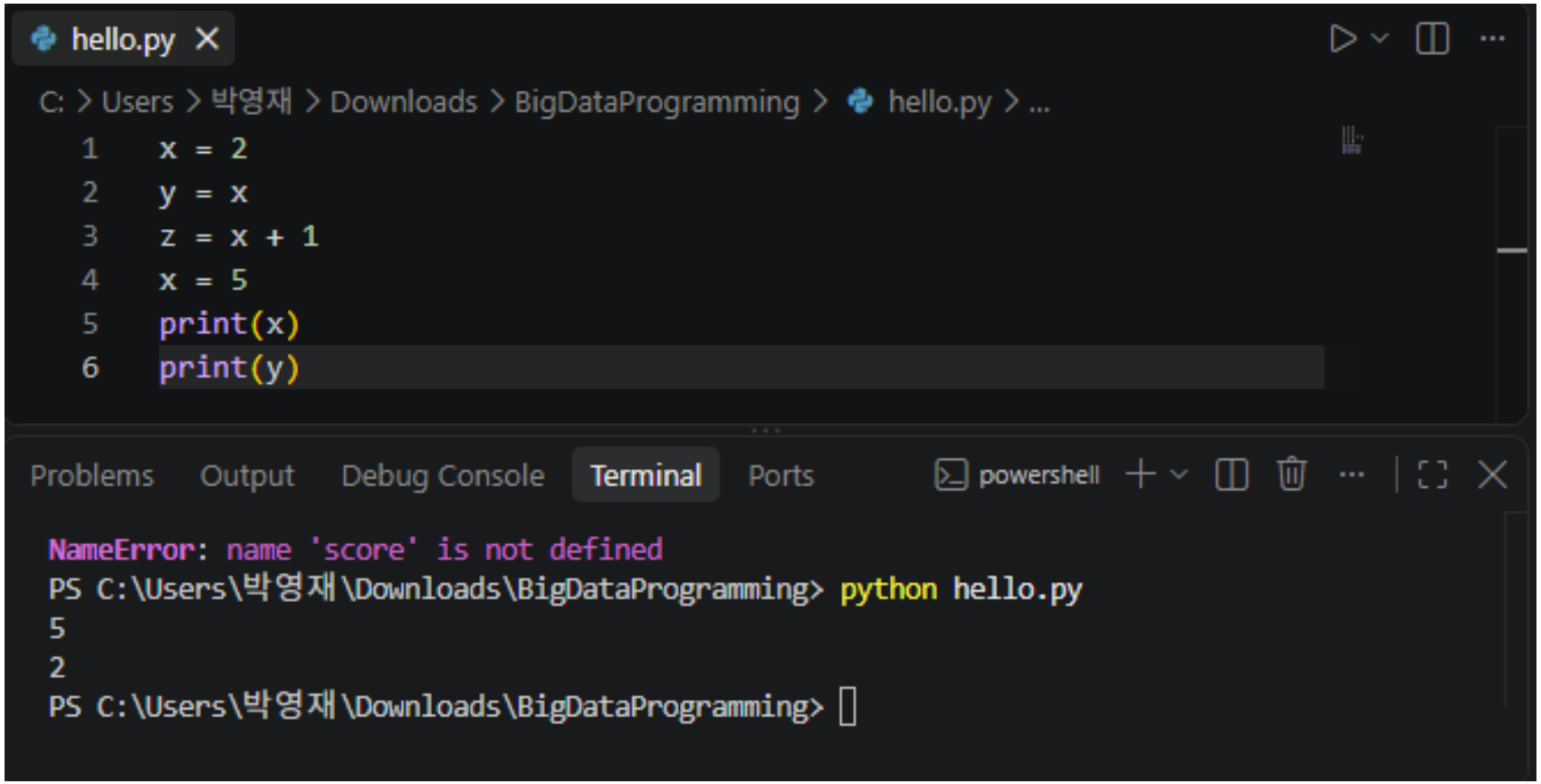
이름	값	값을 얻은 방법
x	2	직접 대입
y	2	현재 x의 값을 참조
z	3	x + 1을 계산

생각해 보기

이후에 x = 5를 실행하면
y와 z도 바뀔까?

Variables

변수의 값 다시 지정하기 (Reassignment)



x = 5는 이름 x가 정수 객체 5를 가리키도록 다시 연결한다.
이전에 x가 가리키던 정수 2 자체를 5로 바꾸는 연산이 아니다.

Variables

변수의 값 다시 지정하기 (Reassignment)

다시 대입하면 이름의 연결이 바뀐다

x = 5는 이름 x가 정수 객체 5를 가리키도록 다시 연결한다. 이전에 x가 가리키던 정수 2 자체를 5로 바꾸는 연산이 아니다. 이 차이가 y의 값이 그대로인 이유를 설명한다.

다른 이름의 연결은 그대로 남는다

y = x를 실행했던 시점에는 x가 2를 가리켰다. 그때 y도 2를 가리키도록 연결되었다. 이후 x를 5에 다시 연결해도 y의 연결은 바뀌지 않는다. z도 앞서 계산한 값 3을 유지한다.

무엇이 바뀌고 무엇이 유지되는가?

현재 예제에서는 x만 다시 대입했다. 따라서 출력 결과는 x가 5, y가 2이다. z를 추가로 출력하면 3이다. 다른 변수를 바꾸려면 그 이름에 대한 별도의 대입이나 연산이 필요하다.

코드 예제

```
x = 2
y = x
z = x + 1
x = 5
print(x)
print(y)
```

실행 결과

```
5
2
```

마지막 상태

x는 5를 가리킨다.
y는 2를 가리킨다.
z는 3을 가리킨다.

Variables

자기 자신의 값 갱신하기와 실행 순서

x = x + 1은 모순된 등식이 아니다

오른쪽의 x는 대입 직전의 값을 뜻한다. 현재 값에 1을 더한 뒤, 그 결과를 왼쪽 x에 다시 대입한다. x가 10이었다면 오른쪽 계산 결과는 11이고 이후 x는 11을 가리킨다.

실행 순서가 달라지면 사용한 값도 달라진다

y = x를 먼저 실행하면 y는 변경 전의 값을 참조한다. x를 갱신한 뒤 y = x를 실행하면 y는 갱신한 값을 참조한다. 코드에 같은 문장들이 있어도 배치 순서가 달라지면 결과가 달라질 수 있다.

오른쪽에서 사용할 이름은 먼저 정의한다

x = x + 1을 처음 실행하는데 x가 아직 정의되지 않았다면 오른쪽 값을 구할 수 없다. 자기 자신의 값을 갱신하는 문장도 먼저 초기값이 필요하다. 대입의 오른쪽부터 읽는 습관으로 확인한다.

갱신 전에 y에 대입

```
x = 10
y = x
x = x + 1
print(x, y)
```

출력: 11 10

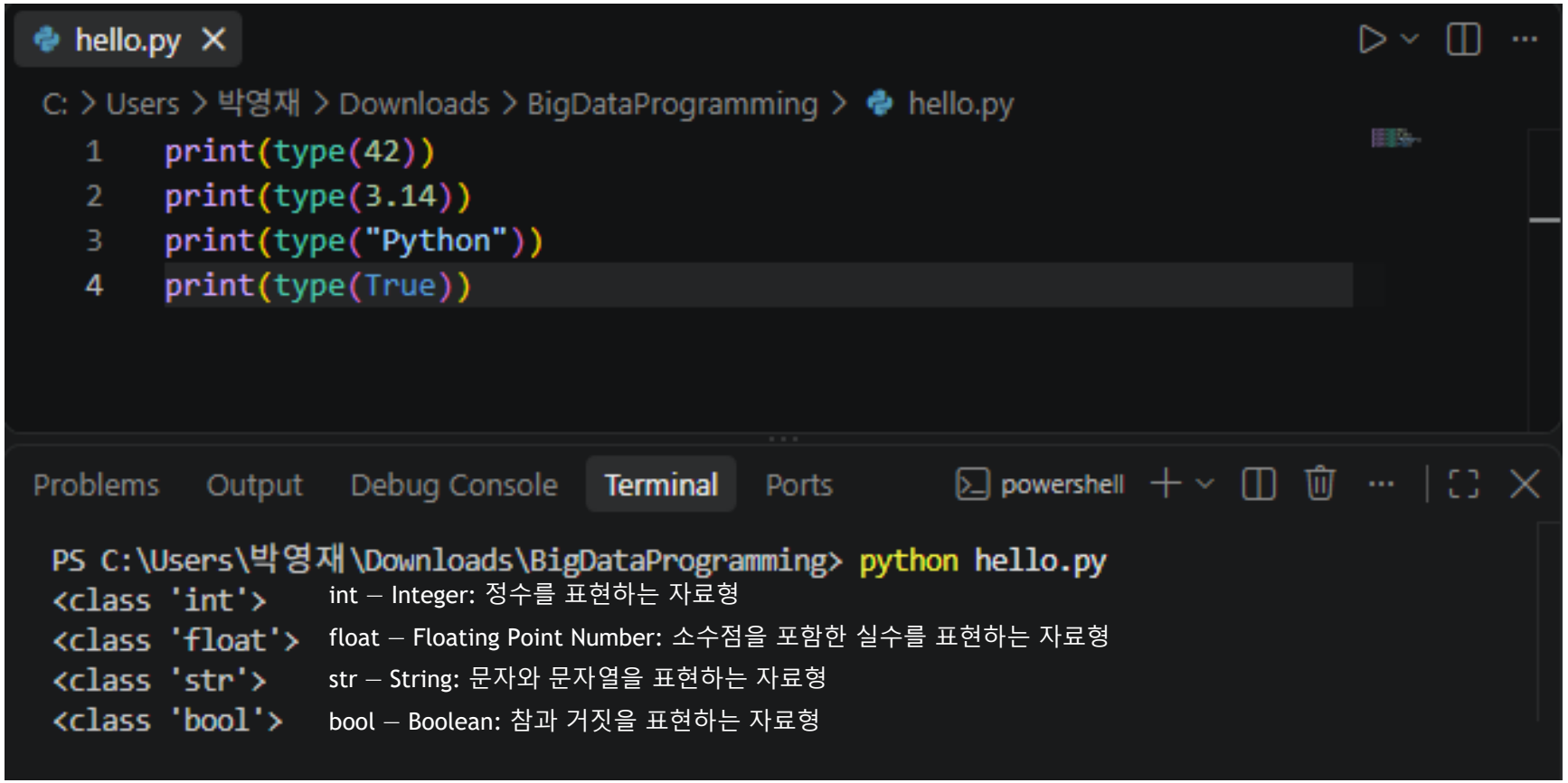
갱신한 뒤 y에 대입

```
x = 10
x = x + 1
y = x
print(x, y)
```

출력: 11 11

Variables

기본 자료형 (Basic Data Types)



type()으로 객체의 자료형을 확인한다.
출력에서 보이는 <class ...>는 객체의 자료형을 나타낸다.

Variables

기본 자료형 (Basic Data Types)

자료형은 값의 종류와 그 값에 적용할 수 있는 연산을 결정한다. 같은 모양의 값이라도 숫자인지 문자열인지에 따라 계산 결과가 달라질 수 있다. 값과 자료형을 함께 확인하는 습관이 필요하다.

자료형	정식 명칭	값의 예	의미
int	Integer	42, 0, -3	정수를 표현한다.
float	Floating Point Number	3.14, -0.5	부동소수점 방식으로 수를 표현한다.
str	String	"Python", "42"	문자열을 표현한다.
bool	Boolean	True, False	참과 거짓을 표현한다.

type()으로 확인하기

print(type(42)), print(type(3.14)), print(type("Python")), print(type(True))를 실행하면 각각 int, float, str, bool 자료형이 표시된다. 출력에서 보이는 <class ...>는 객체의 자료형을 나타낸다.

float는 모든 실수를 정확하게 저장하는 자료형이 아니라 제한된 정밀도로 수를 표현한다. 문자열 "True"는 참을 뜻하는 bool 값 True와 다르다.

Variables

숫자처럼 보여도 문자열일 수 있다

따옴표가 자료형을 바꾼다

10은 정수이지만 "10"은 두 문자로 이루어진 문자열이다. 문자열에 적힌 숫자를 보고 Python이 자동으로 정수로 해석한다고 생각하지 않는다. 숫자로 계산할지 문자로 다룰지 목적에 따라 구분한다.

연산 기호의 의미가 달라진다

10 + 3은 13이지만 "10" + "3"은 "103"이다. 문자열에 정수를 곱하면 문자열이 반복된다. 출력이 비슷해 보여도 실제로 어떤 자료형을 사용했는지 확인해야 결과를 설명할 수 있다.

필요할 때 명시적으로 변환한다

int("10")은 숫자로 해석할 수 있는 문자열을 정수로 변환한다. str(10)은 정수를 문자열로 표현한다. 변환이 가능한 형식이어야 하므로 아무 문자열이나 int()로 바꿀 수 있는 것은 아니다.

연산 결과 비교

```
print(10 + 3)
print("10" + "3")
print("10" * 3)
```

실행 결과

```
13
103
101010
```

변환한 뒤 계산하기

```
print(int("10") + 3)
print(str(10) + "3")
```

출력: 13, 다음 줄에 103

Variables

Dynamic Typing

변수 이름의 자료형을 고정해서 선언하지 않는다

Python에서는 x에 정수를 대입한 뒤 같은 이름에 실수나 문자열을 다시 대입할 수 있다. 동일한 이름이 실행 과정에서 서로 다른 자료형의 객체를 가리킬 수 있다는 뜻이다.

객체 자체의 자료형이 바뀌는 것은 아니다

x = 10 뒤에 x = "Hello"를 실행했다고 정수 10이 문자열 Hello로 변하는 것은 아니다. 이름 x가 다른 객체를 가리키도록 연결이 바뀐다. 자료형은 그때 x가 참조하는 객체를 기준으로 확인한다.

동적 타이핑이 자동 변환을 뜻하지는 않는다

변수 이름을 여러 자료형에 사용할 수 있어도 모든 자료형끼리 계산할 수 있는 것은 아니다. x에 문자열을 대입한 뒤 x + 1을 실행하면 TypeError가 발생할 수 있다. 연산 직전의 자료형을 확인해야 한다.

코드 예제

```
x = 10
print(x, type(x))
x = 3.14
print(x, type(x))
x = "Hello"
print(x, type(x))
```

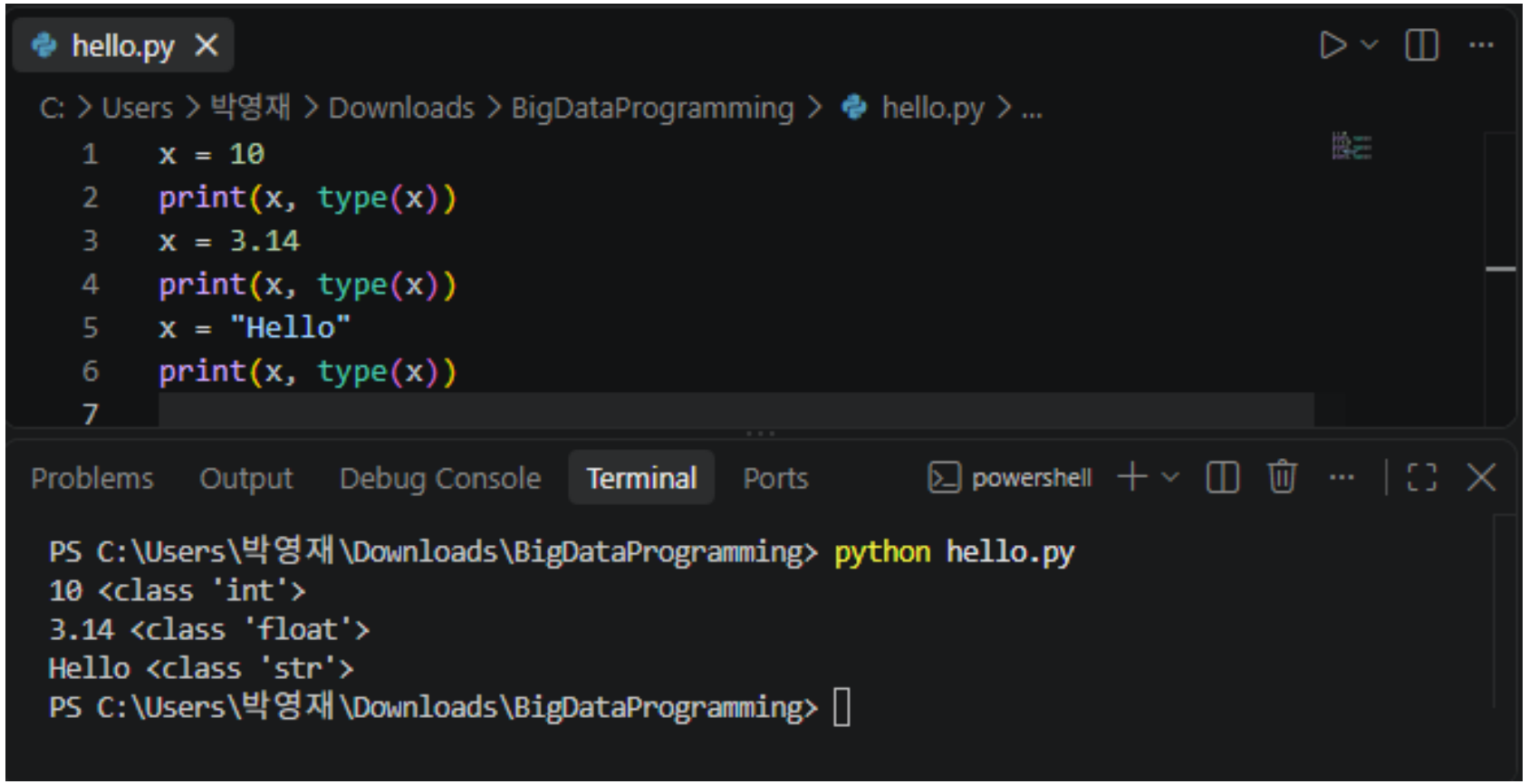
실행 결과

```
10 <class 'int'>
3.14 <class 'float'>
Hello <class 'str'>
```

세 번의 출력은 서로 다른 시점에 x가 가리키는 값과 자료형을 보여 준다.

Variables

Dynamic Typing



이름 x가 다른 객체를 가리키도록 연결이 바뀐다.
자료형은 그때 x가 참조하는 객체를 기준으로 확인한다.

Variables

Multiple Assignment

여러 이름과 값을 순서대로 대응시킨다

x, y = 2, 3은 첫 번째 값 2를 x에, 두 번째 값 3을 y에 대입한다. 현재처럼 별표가 없는 단순한 형태에서는 왼쪽 이름의 개수와 오른쪽에서 전달되는 값의 개수가 맞아야 한다.

오른쪽 값을 먼저 구한다

오른쪽에 계산식이 있으면 필요한 값들을 먼저 평가한 뒤 왼쪽 이름에 대응시킨다. 이 원리를 이해하면 두 변수의 값을 교환하는 x, y = y, x가 왜 가능한지 설명할 수 있다.

짧은 코드보다 대응 관계가 중요하다

여러 값을 한 줄에 대입할 수 있다고 해서 모든 대입을 한 줄로 모을 필요는 없다. 서로 관련 있는 값을 함께 다룰 때 사용하고, 무엇이 어느 이름에 들어가는지 읽기 어렵다면 나누어 작성한다.

코드 예제

```
x, y = 2, 3
print(x, y)
```

출력: 2 3

이 예제에서는 같은 결과

```
x = 2
y = 3
print(x, y)
```

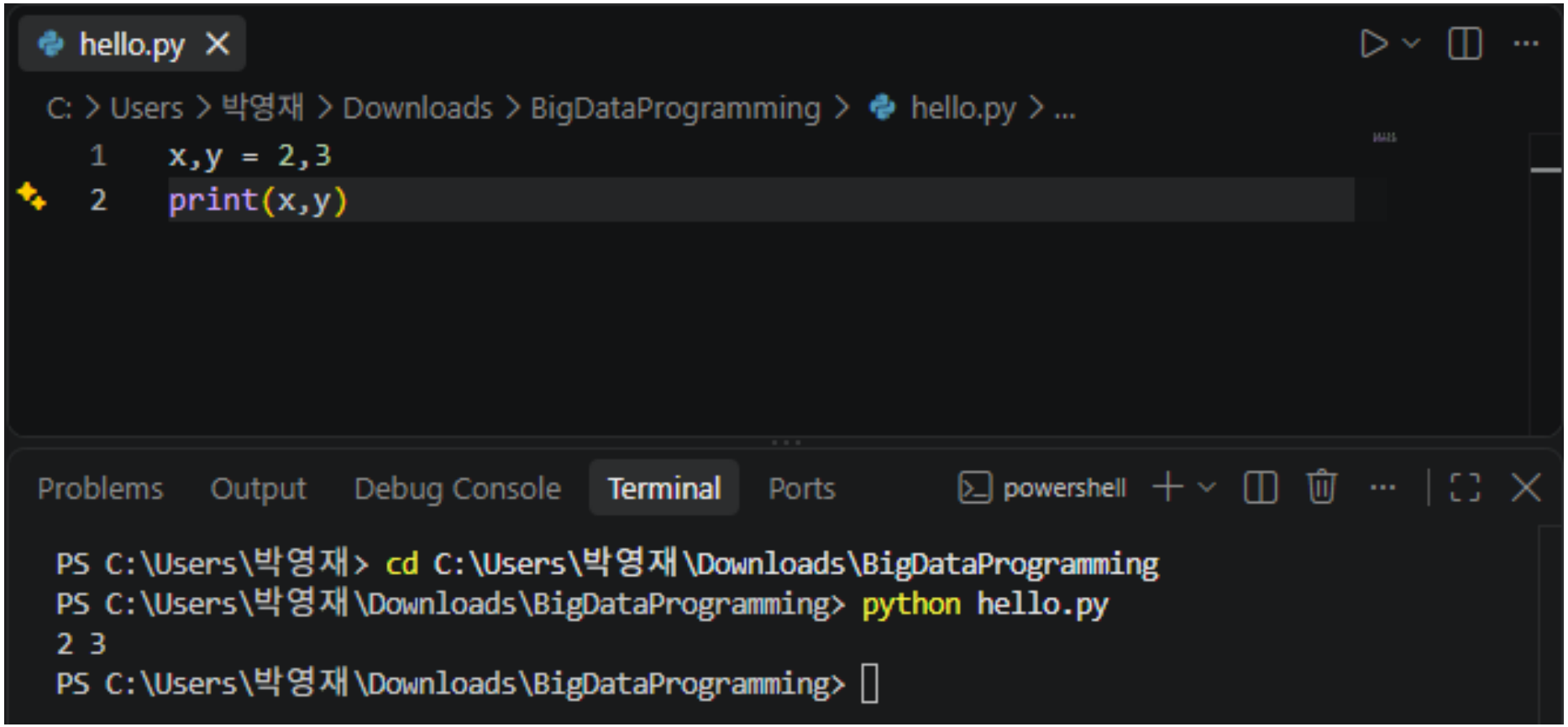
개수가 맞지 않는 예

```
x, y = 2, 3, 4
```

두 이름에 세 값을 대응시킬 수 없어 ValueError가 발생한다.

Variables

Multiple Assignment



x, y = 2, 3은 첫 번째 값 2를 x에, 두 번째 값 3을 y에 대입한다.
무엇이 어느 이름에 들어가는지 확인한다.

Variables

두 변수의 값 교환하기

순차 대입에서는 이전 값이 덮인다

x가 2이고 y가 3일 때 x = y를 실행하면 x도 3을 가리킨다. 이어서 y = x를 실행하면 현재 x의 값 3을 y에 대입한다. 처음 x가 가리키던 2를 다시 가져오는 문장이 아니다.

다중 대입에서는 변경 전의 오른쪽 값을 사용한다

x, y = y, x에서는 대입 전의 y와 x를 먼저 평가한다. 이때 오른쪽 값은 순서대로 3과 2이다. 그 결과를 x와 y에 연결하므로 두 값이 교환된다.

단순 암기보다 대입 순서로 설명한다

왜 첫 번째 코드는 3과 3을 만들고 두 번째 코드는 3과 2를 만드는지, 각 문장이 실행될 때의 값을 적어 본다. 앞으로 코드가 길어져도 같은 방식으로 변수의 상태를 추적할 수 있다.

교환되지 않는 코드

```
x, y = 2, 3
x = y
y = x
print(x, y)
```

출력: 3 3

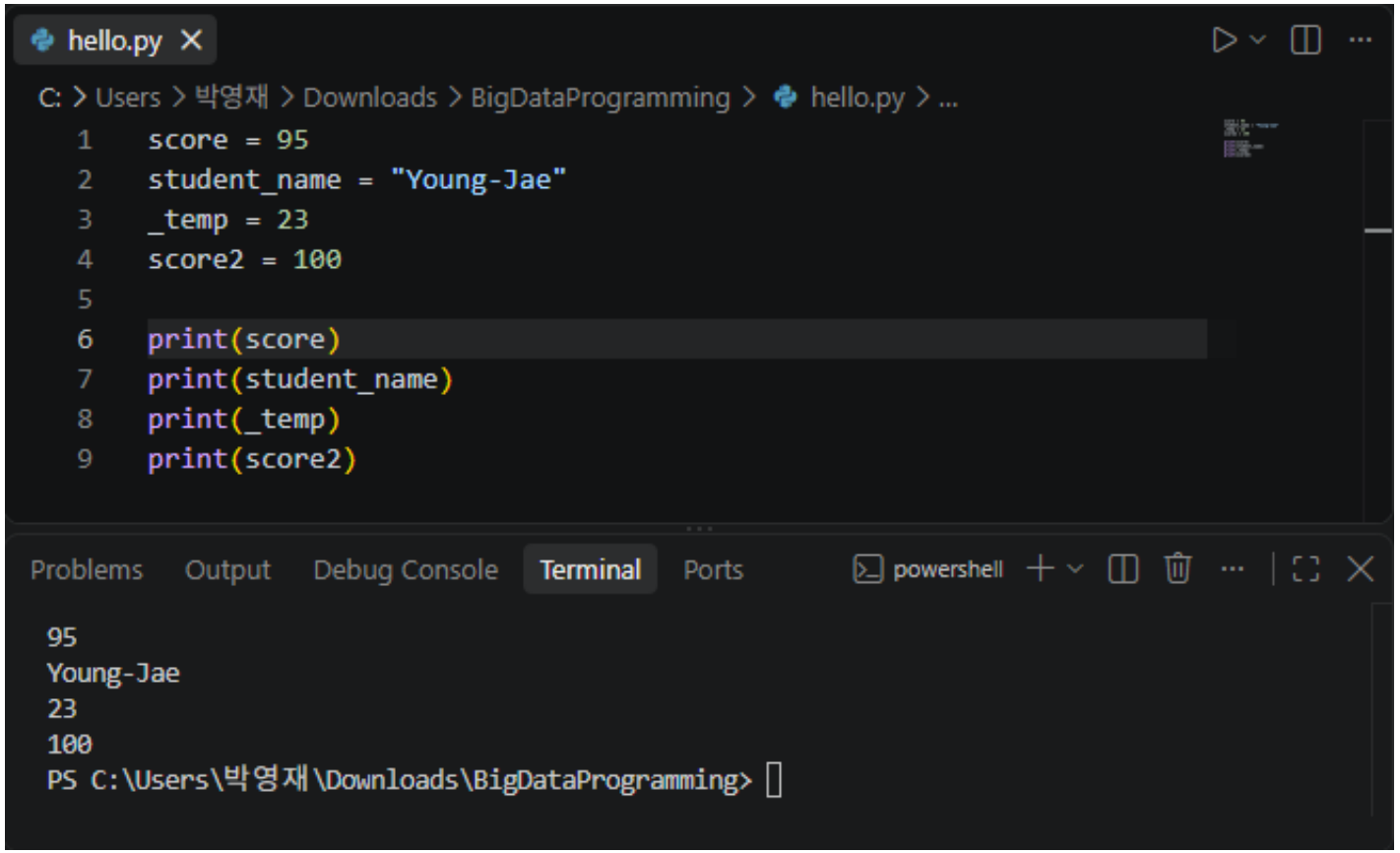
교환되는 코드

```
x, y = 2, 3
x, y = y, x
print(x, y)
```

출력: 3 2

Variables

변수 이름 정하기 (Naming Rules)



문법상 가능한 이름인지와 의미가 좋은 이름인지는 다른 기준이다.
같은 기준을 일관되게 적용한다.

Variables

변수 이름 정하기 (Naming Rules)

문법적으로 사용할 수 있는 이름

이 수업에서는 영문자, 숫자, 밑줄을 조합해 변수 이름을 작성한다. 숫자로 시작할 수 없고 공백을 넣을 수 없다. Python은 일부 비영문 문자도 허용하지만 예제에서는 영문 이름을 일관되게 사용한다.

대소문자와 예약어를 구분한다

score와 Score는 서로 다른 이름이다. class, if처럼 문법에서 특별한 역할을 하는 키워드는 일반 변수명으로 사용할 수 없다. 문법상 가능한 이름인지와 의미가 좋은 이름인지는 다른 기준이다.

이름만으로 값의 역할을 읽을 수 있게 한다

여러 단어는 student_name처럼 소문자와 밑줄로 연결한다. x보다 student_score가 목적을 드러내기 쉽다. 너무 긴 이름이나 의미 없는 번호 나열은 피하고, 같은 기준을 일관되게 적용한다.

현재 예제의 변수명

```
score = 95
student_name = "Young-Jae"
_temp = 23
score2 = 100
```

사용할 수 없는 이름

2score 숫자로 시작한다.
student name 공백이 있다.
class Python 키워드이다.

이름을 읽어 보기

```
student_final_score = 95
```

이름이 길어지는 이유가 값의 역할을 더 명확하게 설명하기 때문인지 확인한다.

Variables

Keywords vs. Built-in Functions

hello.py

Toggle Secondary Side Bar (Ctrl+Alt+B)

C: > Users > 박영재 > Downloads > BigDataProgramming > hello.py > ...

```
1   import keyword
2
3   # Keyword
4   print(keyword.iskeyword("class"))
5   print(keyword.iskeyword("score"))
6
7   # Built-in Function
8   print(sum([1, 2, 3]))
9
10  sum = 100
11  print(sum)
12
13  # print(sum([1, 2, 3]))  # Error!
```

Problems Output Debug Console Terminal Ports powershell

```
PS C:\Users\박영재\Downloads\BigDataProgramming> python hello.py
True
False
6
100
PS C:\Users\박영재\Downloads\BigDataProgramming>
```

sum = 100은 이름 sum을 정수 100에 연결하여 내장 함수를 가린다.
마지막 줄의 주석을 해제하면 정수를 함수처럼 호출하려고 하므로 TypeError가 발생한다.

Variables

Keywords vs. Built-in Functions

Keywords는 문법을 구성하는 단어이다

if, else, for, while, class, return은 문법에서 특별한 의미를 가진 키워드이다. 변수명으로 사용할 수 없다. keyword.iskeyword()로 일반 키워드인지 확인할 수 있다.

Built-in Functions는 기본 제공 함수이다

print(), sum(), max(), len()은 기본 제공 함수이다. 같은 이름에 다른 값을 대입하는 것은 가능하지만, 그 범위에서 원래 함수를 해당 이름으로 사용할 수 없게 될 수 있다.

sum = 100은 무엇을 바꾸는가?

이름 sum을 정수 100에 연결하여 내장 함수를 가린다. 이후 sum([1, 2, 3])을 실행하면 정수를 함수처럼 호출하려고 하므로 TypeError가 발생한다. 합계 변수에는 total 같은 다른 이름을 사용한다.

코드 예제

```
import keyword
print(keyword.iskeyword("class"))
print(keyword.iskeyword("score"))
print(sum([1, 2, 3]))
sum = 100
print(sum)
# print(sum([1, 2, 3]))
```

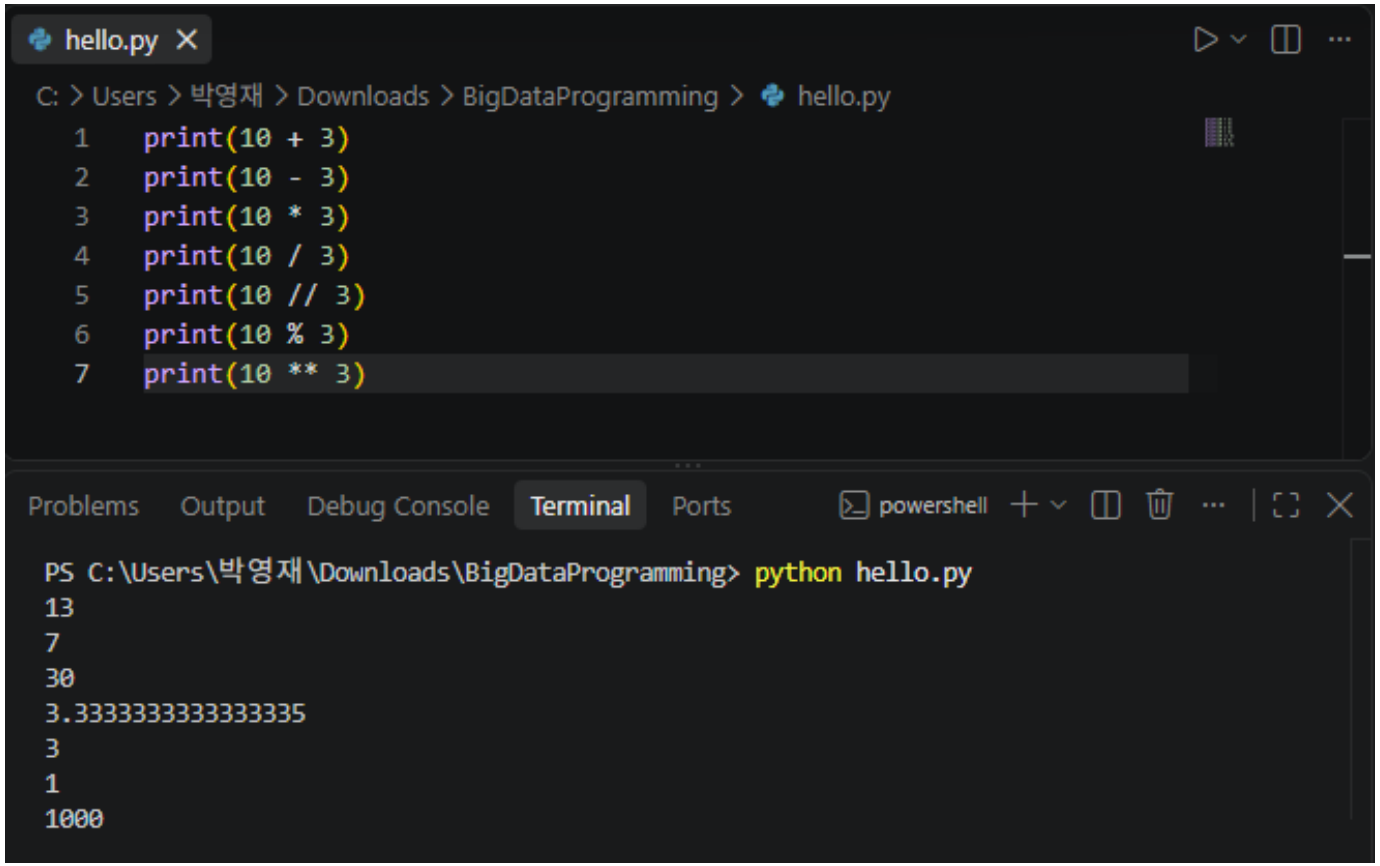
실행 결과

```
True
False
6
100
```

마지막 줄의 주석을 해제하면 sum이 정수이므로 함수 호출에 실패한다.

Expressions

표현식 (Expression)



print(10 + 3)을 실행하면 안쪽 표현식의 결과 13을 출력한다.
계산 결과를 출력할지 저장할지는 바깥 문장의 역할이다.

Expressions

표현식 (Expression)

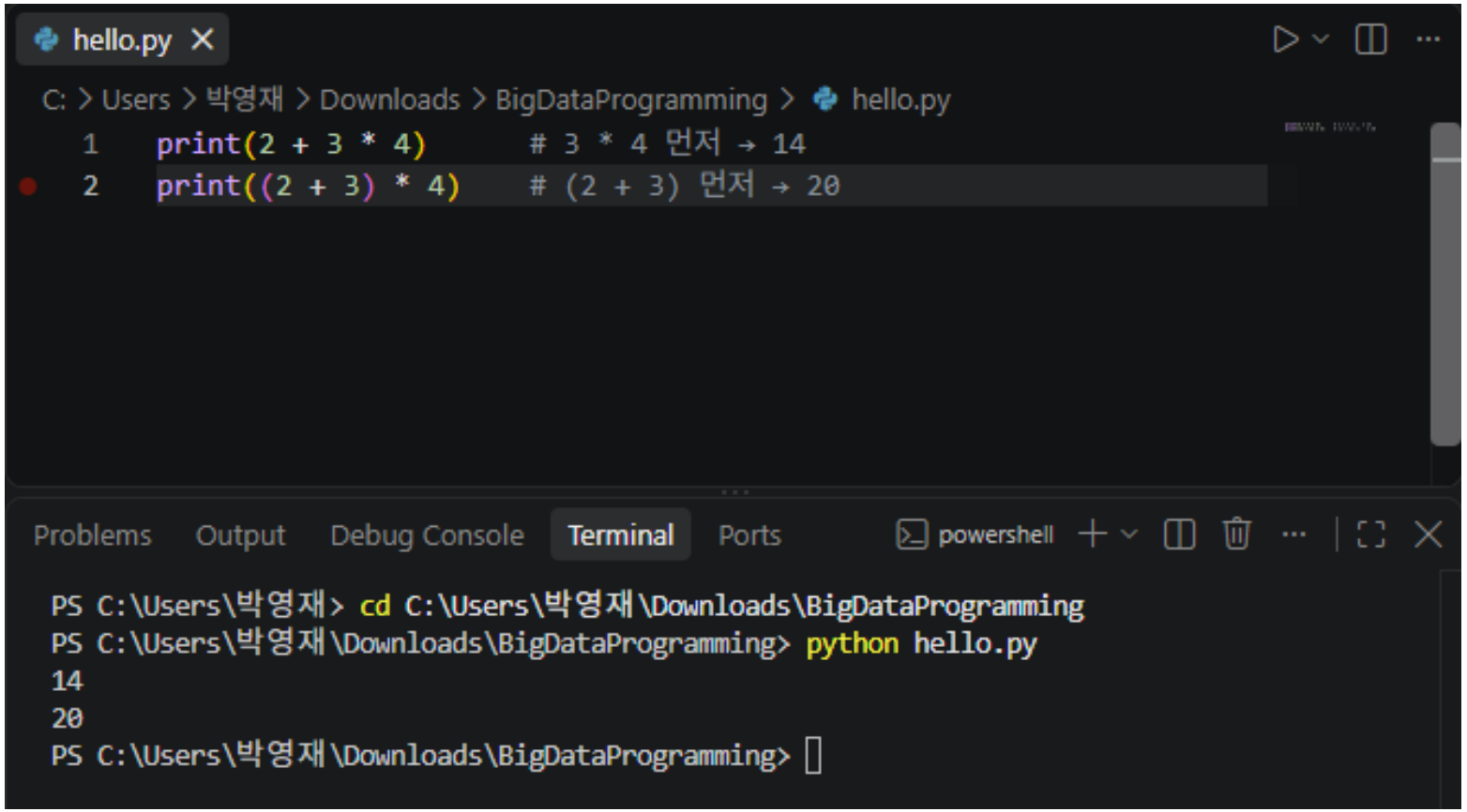
표현식은 평가되어 하나의 값을 만든다. 10, x처럼 값이나 이름만으로 된 표현식도 있고, 10 + 3처럼 연산자를 포함한 표현식도 있다. 10과 3은 피연산자이고 +는 수행할 계산을 나타내는 연산자이다.

연산자	의미	예제	결과
+	덧셈	10 + 3	13
-	뺄셈	10 - 3	7
*	곱셈	10 * 3	30
/	나눗셈	10 / 3	3.3333333333333335
//	내림 나눗셈	10 // 3	3
%	나머지	10 % 3	1
**	거듭제곱	10 ** 3	1000

예를 들어 print(10 + 3)을 실행하면 안쪽 표현식의 결과 13을 출력한다. total = 10 + 3에서는 같은 계산 결과를 이름 total에 대입한다. 계산 결과를 출력할지 저장할지는 바깥 문장의 역할이다.

Expressions

연산 순서 (Operator Precedence)



2 + 3 * 4에서는 곱셈이 덧셈보다 우선한다.
(2 + 3) * 4에서는 괄호 안의 2 + 3을 먼저 계산한다.

Expressions

연산 순서 (Operator Precedence)

모든 연산을 왼쪽부터 하는 것은 아니다

2 + 3 * 4에서는 곱셈이 덧셈보다 우선하므로 3 * 4를 먼저 계산한다. 결과는 2 + 12, 즉 14이다. 식을 적은 순서와 연산자가 적용되는 순서를 구분해야 한다.

괄호로 먼저 계산할 부분을 지정한다

(2 + 3) * 4에서는 괄호 안의 2 + 3을 먼저 계산한다. 따라서 5 * 4가 되어 20을 얻는다. 괄호는 보기 좋게 묶는 표시가 아니라 계산 순서에 영향을 주는 요소이다.

같은 우선순위도 규칙을 확인한다

덧셈과 뺄셈, 곱셈과 나눗셈처럼 같은 우선순위의 기본 연산은 보통 왼쪽부터 묶는다. 거듭제곱은 오른쪽부터 묶이므로 예외이다. 처음에는 의도를 드러내는 괄호를 사용해 읽기 쉬운 식으로 작성한다.

코드 예제

```
print(2 + 3 * 4)
print((2 + 3) * 4)
```

출력: 14, 다음 줄에 20

이번 예제의 우선순위

괄호로 묶은 부분
거듭제곱 **
곱셈과 나눗셈 * / // %
덧셈과 뺄셈 + -

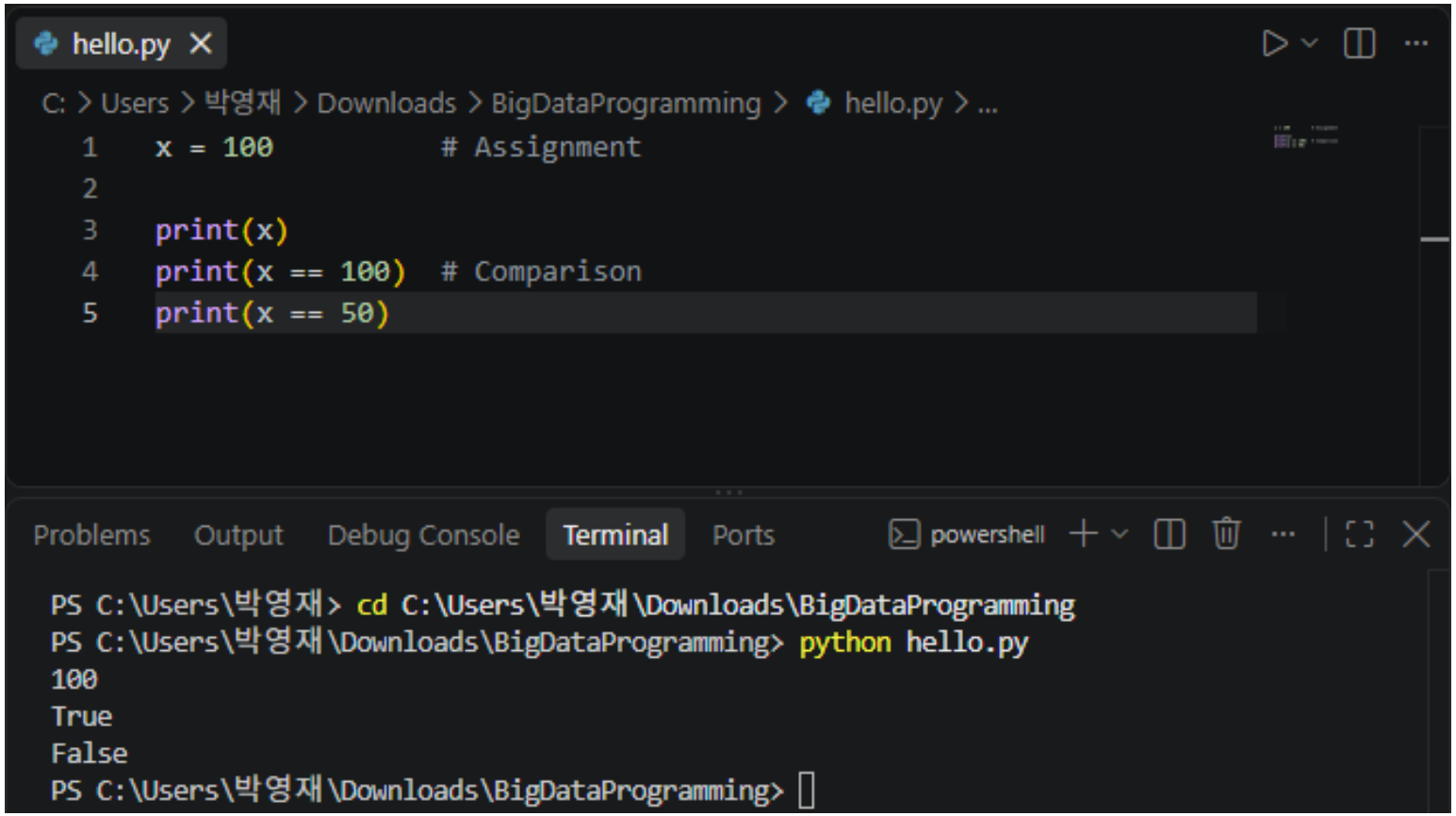
같은 우선순위 확인

```
print(10 - 3 - 2)
```

출력: 5. 먼저 10 - 3을 계산한다.

Expressions

= vs. ==



x = 100은 이름 x를 값 100에 연결하는 문장이다.
x == 100은 현재 x의 값과 100을 비교하는 표현식이다.

Expressions

= vs. ==

=는 이름에 값을 대입한다

x = 100은 이름 x를 값 100에 연결하는 문장이다. 현재 값이 100과 같은지 질문하는 것이 아니다. 새로운 연결을 만들거나 기존 연결을 바꾸므로 이후 코드에서 사용하는 x의 값이 달라질 수 있다.

==는 두 값이 같은지 비교한다

x == 100은 현재 x의 값과 100을 비교하는 표현식이다. 현재 예제처럼 기본 숫자를 비교하면 결과는 True 또는 False이다. 비교했다고 x에 100이 새로 대입되는 것은 아니다.

문장의 역할부터 구분한다

값을 정하려는지, 현재 값을 확인하려는지 먼저 생각한다. x == 50이 False를 만들어도 x는 여전히 100이다. 비교 결과를 사용하려면 출력하거나 다른 이름에 저장하거나 조건으로 활용할 수 있다.

코드 예제

```
x = 100

print(x)
print(x == 100)
print(x == 50)
```

실행 결과

```
100
True
False
```

두 기호의 역할

=는 대입한다.
==는 같은지 비교한다.

Expressions

비교 연산자로 조건 만들기

비교 연산자는 두 값의 관계를 확인한다. 현재의 숫자 예제에서는 비교 결과가 bool 값이다. 특히 경계값을 포함할지 제외할지에 따라 >와 >=, <와 <=를 구분해서 작성해야 한다.

연산자	의미	age = 20일 때	결과
==	같다	age == 20	True
!=	다르다	age != 20	False
>	크다	age > 20	False
<	작다	age < 20	False
>=	크거나 같다	age >= 20	True
<=	작거나 같다	age <= 20	True

조건의 문장을 코드로 옮기기

“20세 이상”은 20을 포함하므로 age >= 20이다. “20세 초과”는 20을 제외하므로 age > 20이다. 조건이 맞는지 확인할 때는 기준보다 작은 값, 같은 값, 큰 값을 각각 넣어 보면 좋다.

Expressions

Boolean Expressions

hello.py

C: > Users > 박영재 > Downloads > BigDataProgramming > hello.py > ...

```
1  age = 20
2  is_student = True
3
4  print(age >= 18 and is_student)
5  print(age < 18 or is_student)
6  print(not is_student)
```

Problems Output Debug Console Terminal Ports powershell

```
PS C:\Users\박영재> cd C:\Users\박영재\Downloads\BigDataProgramming
PS C:\Users\박영재\Downloads\BigDataProgramming> python hello.py
True
True
False
PS C:\Users\박영재\Downloads\BigDataProgramming>
```

이 예제에서는 비교 결과와 bool 값만 결합하므로 결과도 True 또는 False이다.
일반적인 and와 or는 피연산자 자체를 반환할 수 있다.

Expressions

Boolean Expressions

and는 두 조건을 함께 만족하는지 확인한다

age >= 18 and is_student는 성인이라는 조건과 학생이라는 조건을 모두 만족하는지 확인한다. age가 20이고 is_student가 True이면 두 조건이 모두 참이므로 결과는 True이다.

or는 적어도 하나의 조건을 만족하는지 확인한다

age < 18 or is_student에서 나이 조건은 False이지만 학생 조건은 True이다. 따라서 전체 결과는 True이다. 두 조건이 동시에 참인 경우에도 or의 결과는 참이다.

not은 조건의 참과 거짓을 반대로 바꾼다

not is_student는 is_student의 값이 True일 때 False이다. 이 예제에서는 비교 결과와 bool 값만 결합하므로 결과도 True 또는 False이다. 일반적인 and와 or는 다른 자료형에 적용하면 피연산자 자체를 반환할 수 있다.

코드 예제

```
age = 20
is_student = True

print(age >= 18 and is_student)
print(age < 18 or is_student)
print(not is_student)
```

실행 결과

```
True
True
False
```

세 연산의 의미

and	두 조건 모두 만족
or	한 조건 이상 만족
not	조건의 참과 거짓 반전

Expressions

복합 조건과 단축 평가 이해하기

각각의 비교 조건을 분명하게 작성한다

18세 이상이고 25세 이하인지는 age >= 18 and age <= 25로 확인한다. age >= 18 and 25라고 적으면 두 번째 부분이 나이를 비교하는 조건이 되지 않는다.

뒤의 계산이 생략되는 단축 평가

and는 앞부분이 거짓이면 뒤를 평가할 필요가 없다. or는 앞부분이 참이면 뒤를 평가할 필요가 없다. 이를 단축 평가라고 한다. 오른쪽 예제에서는 count가 0이면 나눗셈을 실행하지 않는다.

and와 or가 항상 bool을 반환하는 것은 아니다

0 or 5는 5, "Python" and 3은 3을 반환한다. True나 False가 필요하면 조건의 형태와 자료형을 확인한다. 처음에는 비교식과 bool 값을 결합해 의미가 분명한 조건을 작성한다.

범위 조건

```
age = 20
print(age >= 18 and age <= 25)
```

출력: True

뒤의 계산이 생략되는 경우

```
count = 0
print(count != 0 and 10 / count > 2)
```

출력: False. 나눗셈은 실행하지 않는다.

bool 이외의 결과

```
print(0 or 5)
print("Python" and 3)
```

출력: 5, 다음 줄에 3

Practice

Predict Before You Run

먼저 실행하지 않고 값을 추적한다

첫 줄부터 한 줄씩 읽고 x, y, z가 어떤 값을 가리키는지 적는다. 대입문에서는 오른쪽의 현재 값을 먼저 구하고, 왼쪽 이름의 값을 갱신한다. 처음에 없던 이름은 대입된 시점부터 사용한다.

마지막 출력문까지 계산한다

print()가 실행되는 시점의 값을 기준으로 결과를 적는다. 마지막 줄에서는 x > 10과 y < 5를 각각 계산한 뒤 and의 결과를 구한다. 변수의 최종 값과 출력되는 값을 구분하여 확인한다.

예상한 뒤 실행하고 차이를 설명한다

VS Code에서 같은 코드를 실행하고 예상 결과와 비교한다. 결과가 다르면 틀린 답만 지우지 말고 어느 줄에서 다른 값을 사용했는지 찾는다. 맞혔더라도 왜 그 값이 나오는지 말로 설명해 본다.

실습 코드

```
x = 10
y = 3

z = x // y
x = x + 1

print(x)
print(y)
print(z)
print(x > 10 and y < 5)
```

예상 결과

x는 얼마인가? y는 얼마인가? z는 얼마인가? 마지막 조건은 참인가?

Practice

Predict Before You Run

hello.py

C: > Users > 박영재 > Downloads > BigDataProgramming > hello.py > ...

```
1   x = 10
2   y = 3
3
4   z = x // y
5   x = x + 1
6
7   print(x)
8   print(y)
9   print(z)
10  print(x > 10 and y < 5)
```

x = ?
y = ?
z = ?
Boolean = ?

대입문에서는 오른쪽의 현재 값을 먼저 구하고, 왼쪽 이름의 값을 갱신한다.
예상한 뒤 실행하고 차이가 있다면 그 이유를 설명한다.

Practice

실행 결과와 변수의 변화 설명하기

z를 계산할 때는 x가 아직 10이다. 따라서 z는 10 // 3의 결과인 3을 가리킨다.
이후 x를 11로 갱신해도 z를 다시 계산하는 문장은 없으므로 z는 그대로 3이다.

실행한 문장	x	y	z
x = 10	10	아직 없음	아직 없음
y = 3	10	3	아직 없음
z = x // y	10	3	3
x = x + 1	11	3	3

최종 출력은 11, 3, 3, True이다

마지막 조건에서 11 > 10과 3 < 5는 모두 True이다. and로 연결한 결과도 True이다.
갱신된 x는 마지막 비교에 사용되지만, 앞서 계산한 z를 자동으로 갱신하지는 않는다.

같은 원리를 다른 값에도 적용한다

x = x + 1 대신 x = 20을 실행해도 z는 3을 유지한다. 이후 z = x // y를 다시 실행해야
z가 6으로 바뀐다. 답을 외우기보다 계산 시점과 재대입 여부로 설명한다.

Introduction to Python

Chapter Overview

Development Environment

Python 인터프리터는 코드를 실행하고 VS Code는 파일 작성과 관리를 돕는다. 코드를 바꾼 뒤에는 저장한 파일과 실행한 파일이 같은지 확인한다. Interactive Mode의 결과 표시와 .py 파일의 실행 방식도 구분한다.

Running Python

print()로 값과 결과를 확인하고 주석으로 코드의 목적을 설명한다. 문법 오류와 실행 중 발생하는 예외를 구분하며, 오류 종류와 발생 위치를 찾아 원인을 좁힌다. 정상 종료와 올바른 결과는 같은 뜻이 아니다.

Python Basics

변수는 객체를 가리키는 이름이며, 대입은 오른쪽 값을 먼저 구한 뒤 연결한다. 자료형에 따라 가능한 연산이 달라지고, 연산 우선순위와 실행 순서가 결과에 영향을 준다. 비교식과 논리 연산으로 조건을 구성할 수 있다.

다음 단계로 연결하기

다음에는 조건의 결과에 따라 다른 코드를 실행하는 방법을 배운다. 먼저 결과를 예상하고, 실행 후에는 그 이유를 설명하는 습관을 유지한다.